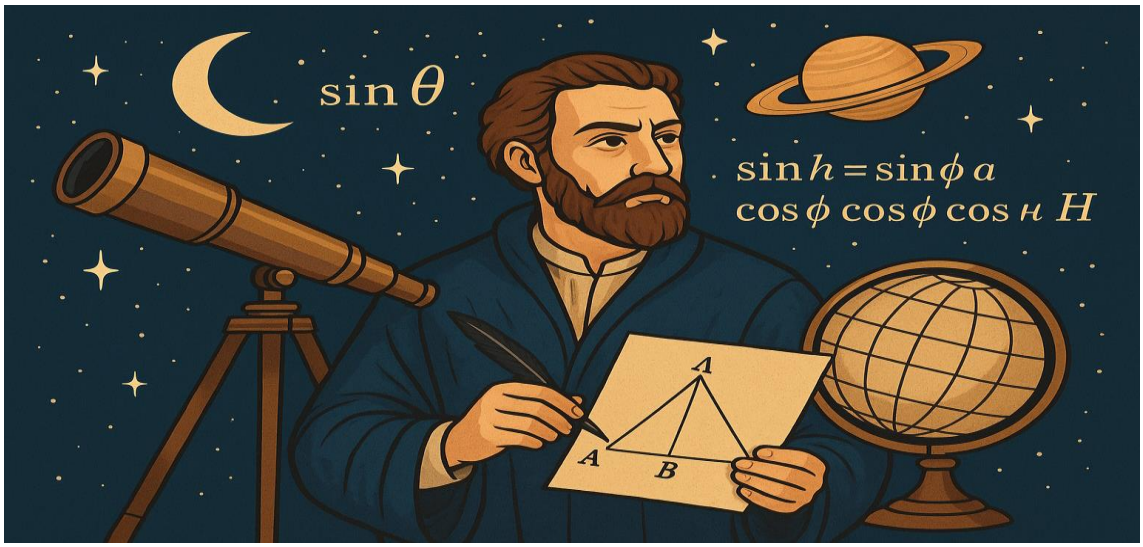
Aproximări ale funcțiilor elementare în hardware:

ARCTAN

Ceuță Rareș




# Contex.

Elementul fundamental al științei moderne este aparatul matematic. Prin matematică reușim să descriem lumea într-un mod precis și inteligibil. Aparatele de măsură colectează o cantitate impresionantă de date care trebuie procesate matematic. Astfel apare necesitatea unor sisteme de calcul care să fie capabile să execute un set mare calcule ale diferitor funcții matematice.

Scopul acestui proiect este acela de a prezenta metode de calcul ale funcției arctangenta în hardware, prezentându-se totodată performanța fiecarei metode propuse.

# Obiective.

Unitatea de calcul pentru funcția arctangentă va fi proiectată în limbajul VHDL. De asemenea vom prezenta o simulare pentru fiecare metodă de calcul implementată și grafice privind performanța.

Metodele de calcul sunt:

1.Abordare bazată pe look-up tables.

Această metodă se bazează pe o memorie ROM care conține un set de valori pentru arctan, preluate din tabele de valori. Atunci când vrem să calculăm valoarea funcției arctan(x) vom căuta în tabel cea mai apropiată valoare de x, notată xi. În tabel avem asociată valorii xi, valoarea funcției arctan(xi). În această situație există două abordări. Prima și cea mai simplă este de a afișa valoarea arctan(xi). A doua este de a căuta valoarea arctan(xi-1) și de a efectua o interpolare între cele două valori, afișând rezultatul interpolării.

2.Metode shift-and-add. În hardware operațiile de înmulțire sunt costisitoare din punct de vedere al performanței. Din acest motiv, o metodă de calcul bazată pe adunări și deplasări pe biți este mult mai eficientă decât o metodă care folosește operația de înmulțire. Funcția arctan poate fi obținută prin rotații succesive de vectori în planul 2D, fapt care ne ajută să obținem valoarea funcției folosind doar operații de adunare și deplasare pe biți. Această metodă stă la baza algoritmului CORDIC pentru calcul al funcțiilor trigonometrice. Vom prezenta metoda precisă de implementare în capitolul dedicat acestui subiect.

3. Tehnologia utilizată

VHDL - Limbaj standardizat utilizat pentru descrierea arhitecturii hardware.

**FPGA Xilinx** - Ținta implementării este un FPGA din familia Xilinx, utilizând fluxul standard de dezvoltare oferit de Vivado. Alegerea acestei platforme este justificată de disponibilitatea instrumentelor profesionale de sinteză, implementare și analiză a performanței.

**AXI4-Stream -** Deși implementarea prezentă este un modul izolat, proiectul este compatibil conceptual cu integrarea într-un sistem bazat pe interfața AXI4-Stream, utilizată frecvent pentru procesarea de date în flux continuu.

**Biblioteci:**

IEEE.STD_LOGIC_1164.ALL; - reprezentare valori

IEEE.NUMERIC_STD.ALL

std.textio.all; - pentru afișare rezultate în consolă

ieee.std_logic_textio.all;

IEEE.MATH_REAL

Pentru documentare am folosit informații din cartea Digital Processing with Field Programmable Arrays scrisă de Uwe Meyer Baese. A fost folositoare pentru documentarea și implementarea algoritmului cordic în modul vectorial.

## 4. Arhitectură

**4.1**Această secțiune descrie în detaliu **arhitectura implementării LUT**, corespunzătoare modulului atan_lut.

Modulul este definit prin următoarea interfață sincronă:

- `clk` – semnal de ceas pentru sincronizarea întregii arhitecturi
- `val_in` (32 biți, unsigned) – valoarea de intrare, reprezentând argumentul funcției arctan, scalat pe domeniul [0, 1]

- `angle_out` (32 biți, unsigned) – valoarea aproximată a funcției arctan, scalată pe domeniul [0, π/4]

Interfața este compatibilă cu integrarea ulterioară într-o arhitectură de tip streaming (AXI4Stream), deoarece modulul produce un rezultat la fiecare ciclu de ceas după umplerea pipeline-ului.

### 4.1.1Reprezentare Numerică

Pentru a calcula rezultatul arctan avem nevoie de o reprezentare numerică a rezultatului care să fie compatibilă cu posibilitățile de reprezentare pe fpga. Astfel am ales o mapare între intervalul 0-2^29 și intervalul 0-π/4. În tabela LUT avem 1024 de valori fapt care înseamnă că am împărțit intervalul 0-2^29 în 1024 de părți egale. Pentru a transforma o valoare din intervalul 0-2^29 se înmulțește rezultatul cu un factor de scalare. Altfel spus **valoare_reală = valoare_lut / 2^29**. Aceasta este metoda aleasă de reprezentare. Pentru îmbunătățirea preciziei fără creșterea dimensiunii memoriei, se aplică o tehnică de **interpolare liniară** între două valori consecutive din tabel.

### 4.1.2Pași de execuție:

Pentru o valoare de intrare `x`, algoritmul hardware realizează următorii pași:

1. determinarea indexului în tabel (cei mai semnificativi 10 biți ai lui `val_in`);
2. citirea valorilor `y0 = LUT[i]` și `y1 = LUT[i+1]`;
3. calculul pantei locale: `dy = y1 - y0`;
4. utilizarea părții fracționare (`dx`) pentru interpolare;
5. calculul rezultatului final: `y = y0 + dx * dy`.

Această abordare permite obținerea unei precizii superioare unei LUT simple, cu un cost hardware moderat.

Arhitectura este organizată sub forma unui pipeline sincron pe 4 etape, implementat într-un singur proces secvențial.

**Etapa 1 -** se separă valoarea de intrare în două componente:

addr_i – indexul LUT (biții 31…22 din val_in);
dx – componenta fracționară utilizată pentru interpolare (biții 21…0).

Se calculează și adresa următoare addr_next, necesară pentru interpolare.

```
if rising_edge(clk) then
    --in acesta etapa separam bitii de adresa
    --cei mai semnificativi 10 biti din valoarea de intrare
    --vor reprezenta indexul
    addr_i <= to_integer(val_in(31 downto 22));

    --regula de obtinere a adresei urmatoare celei curente
    if to_integer(val_in(31 downto 22)) < (rom_depth -1) then
        addr_next <= to_integer (val_in(31 downto 22)) + 1;
    else
        addr_next <= rom_depth - 1;
    end if;
```

**Etapa 2 – Accesul la memoria LUT**

În această etapă se citesc valorile corespunzătoare celor două adrese:

y0 = LUT_ROM(addr_i)
y1 = LUT_ROM(addr_next)

Memoria LUT este definită static în VHDL ca un vector constant de 1024 elemente a câte 32 biți

```
--etapa 2 - citirea din rom
y0 <= lut_rom(addr_i);
y1 <= lut_rom(addr_next);
```

Etapa 3 – Calculul diferenței și al produsului

Se calculează diferența:

dy = y1 - y0

Apoi se realizează produsul dintre fracțiune și pantă:

product = dx * dy

Produsul are o lățime extinsă de 54 biți pentru a evita pierderea de precizie.

```
--etapa 3 - calculul valorii de interpolare
if y1 >= y0 then
    dy <= y1 - y0;
else
    dy <= (others => '0');
end if;

product <= dx * dy;
```

Etapa 4 – Interpolarea și generarea rezultatului

Din produs este extrasă partea semnificativă:

interpol = product(53 downto 22)

Rezultatul final este calculat prin:

angle_out = y0 + interpol.

Astfel, rezultatul este disponibil la ieșire după umplerea pipeline-ului.

### 4.1.4. Avantaje:

- latență mică;
- structură regulată și ușor de înțeles;
- precizie bună datorită interpolării;
- comportament determinist și stabil.

### 4.1.5. Limitări:

- consum de memorie (ROM de 1024 × 32 biți);
- utilizarea unui multiplicator hardware pentru interpolare;
- scalabilitate limitată dacă se dorește creșterea drastică a rezoluției.

## 4.1.6 Rezultate simulare:

```
----------------------------------------
Test 1: (0.0)
  X Input         : 0.000000e+00
  Val_in (Hex)    : 00000000
  angle_out (Hex)    : 00000000
  LUT Radiani     : 0.000000e+00
  MATH Radiani    : 0.000000e+00
  Eroare (abs)    : 0.000000e+00
----------------------------------------
Test 2: (1.0)
  X Input         : 1.000000e+00
  Val_in (Hex)    : FFFFFFFF
  angle_out (Hex)    : 200A3223
  LUT Radiani     : 7.863757e-01
  MATH Radiani    : 7.853982e-01
  Eroare (abs)    : 9.775145e-04
----------------------------------------
Test 3: (0.5)
  X Input         : 5.000000e-01
  Val_in (Hex)    : 80000000
  angle_out (Hex)    : 12E818F8
  LUT Radiani     : 4.640385e-01
  MATH Radiani    : 4.636476e-01
  Eroare (abs)    : 3.909303e-04
----------------------------------------
```

```
----------------------------------------
Test 4: (0.123)
  X Input         : 1.230000e-01
  Val_in (Hex)    : 1F7CED90
  angle_out (Hex)    : 04FBF4E7
  LUT Radiani     : 1.223308e-01
  MATH Radiani    : 1.223853e-01
  Eroare (abs)    : 5.446979e-05
----------------------------------------
Test 5: (0.888)
  X Input         : 8.880000e-01
  Val_in (Hex)    : E353F7D0
  angle_out (Hex)    : 1D9C63BE
  LUT Radiani     : 7.267608e-01
  MATH Radiani    : 7.261456e-01
  Eroare (abs)    : 6.151827e-04
----------------------------------------
Test 6: (0.009)
  X Input         : 9.000000e-03
  Val_in (Hex)    : 024DD2F0
  angle_out (Hex)    : 005CFD93
  LUT Radiani     : 8.915355e-03
  MATH Radiani    : 8.999757e-03
  Eroare (abs)    : 8.440215e-05
========================================
```

## 4.6 Metoda CORDIC

**4.6.1** Algoritmul CORDIC (COordinate Rotation DIgital Computer) este o metodă iterativă de calcul a funcțiilor trigonometrice și inversele acestora folosind exclusiv operații de tip shift, add și subtract, ceea ce îl face extrem de eficient pentru implementare hardware pe FPGA.

În modul de operare **vectoring mode**, utilizat în acest proiect, algoritmul pornește de la un vector inițial (x, y) și aplică o serie de rotații elementare astfel încât componenta y să tindă către zero. În urma acestui proces, valoarea acumulată a unghiului reprezintă aproximarea funcției.

### 4.6.2 Reprezentare Numerică

Pentru a calcula rezultatul arctan avem nevoie de o reprezentare numerică a rezultatului care să fie compatibilă cu posibilitățile de reprezentare pe fpga. Astfel am ales o mapare între intervalul 0-2^29 și intervalul 0-π/4. În tabela LUT avem 1024 de valori fapt care înseamnă că am împărțit intervalul 0-2^29 în 1024 de părți egale. Pentru a transforma o valoare din intervalul 0-2^29 se înmulțește rezultatul cu un factor de scalare. Altfel spus **valoare_reală = valoare_lut / 2^31**. Aceasta este metoda aleasă de reprezentare.

### 4.6.3

#### Structura generală a modulului cordic_final

Modulul este compus din următoarele blocuri funcționale:

- Interfață de control: semnalele clk, start, ready, busy
- Registre de stare pentru variabilele interne: x, y, z
- Contor de iterații: i
- Memorie ROM pentru constantele arctan($2^{-i}$): atan_rom
- Bloc de calcul iterativ: operații shift + add/subtract
- Bloc de normalizare a cadranelor (pre-procesare a intrării)

### 4.6.4Tratarea cadranelor (pre-procesare vector)

Înainte de începerea iterațiilor CORDIC, se realizează o normalizare a vectorului de intrare (x_in, y_in) astfel încât acesta să fie adus în cadranul I.

Implementarea realizează:

- dacă x ≥ 0 → cadranele I și IV → nu se modifică vectorul
- dacă x < 0 și y ≥ 0 → cadranul II → rotație inițială cu -90°
- dacă x < 0 și y < 0 → cadranul III → rotație inițială cu +90°

Această etapă permite algoritmului CORDIC să funcționeze corect pe întreg domeniul [-π, π].

```
if start = '1' and busy = '0' then
    if x_in >= 0 then
        --cadranele 1, 4
        x <= x_in;
        y <= y_in;
        z <= (others => '0');
     elsif  (y_in >= 0) then
            --cadranul 2
            --rotim vectorul cu -90 de grade pentru a ajunge in cadranul 1
            x <= y_in;
            y <= -x_in;
            z <= ang_90;
      else
            --cadranul 3
            --rotim vectorul cu +90 de grade pentru a ajunge in cadranul 1
            x <= -y_in;
            y <= x_in;
            z <= -ang_90;
      end if;
      i <= 0;
      busy <= '1';
      ready <= '0';
```

### Blocul iterativ CORDIC

După inițializare, modulul intră în starea busy = '1' și execută iterativ:

- deplasări aritmetice:
  - x_shift = x >> i
  - y_shift = y >> i
- decizie pe baza semnului lui y:
  - dacă y ≥ 0:
    - x ← x + y_shift
    - y ← y - x_shift
    - z ← z + atan_rom(i)
  - altfel:
    - x ← x - y_shift
    - y ← y + x_shift
    - z ← z - atan_rom(i)

Fiecare iterație rafinează aproximația unghiului, iar precizia crește proporțional cu numărul de iterații.

În implementare:

- numărul de iterații = 31
- precizia unghiului este aproximativ de ordinul a $2^{-31}$ radiani

### 4.6.5Control secvențial și latență

Controlul execuției este realizat prin:

- semnalul start – declanșează o nouă conversie
- semnalul busy – indică faptul că algoritmul este în execuție
- semnalul ready – indică finalizarea calculului

Caracteristici temporale:

- 1 ciclu pentru inițializare
- 31 cicluri pentru iterații
- latență totală ≈ 32 cicluri de ceas per rezultat

**Avantaje:**

- Nu necesită multiplicatori hardware
- Consum redus de resurse
- Scalabilitate simplă prin creșterea numărului de iterații
- Precizie controlabilă prin parametrul iterations

**Dezavantaje:**

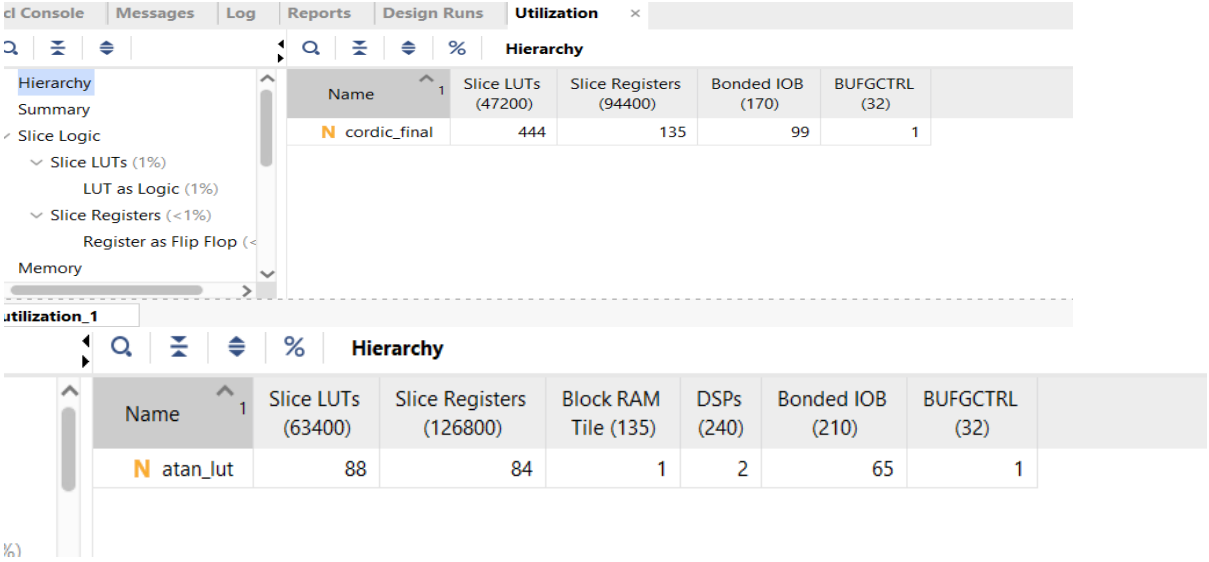
- Latență ridicată
- Debitul este limitat la un rezultat per N cicluri, unde N reprezintă numărul de iterații.

### 4.6.6 Rezultate CORDIC

```
Test 4: cadranul 4 (-45 grade)
  X=10000 Y=-10000
  CORDIC Rad: -7.853532e-01
  REAL   Rad: -7.853982e-01
  Eroare   : 4.496718e-05
----------------------------------------
TEST 5: cadranul 1 (aprox 90 grade)
  X=100 Y=10000
  CORDIC Rad: 1.560833e+00
  REAL   Rad: 1.560797e+00
  Eroare   : 3.659184e-05
----------------------------------------
SIMULARE FINALIZATA COMPLET
----------------------------------------
```

```
SIMULARE FINALIZATA COMPLET
----------------------------------------
Test 1: cadranul 1 (45 grade)
  X=10000 Y=10000
  CORDIC Rad: 7.854431e-01
  REAL   Rad: 7.853982e-01
  Eroare   : 4.496718e-05
----------------------------------------
Test 2: cadranul 2 (135 grade)
  X=-10000 Y=10000
  CORDIC Rad: 2.356239e+00
  REAL   Rad: 2.356194e+00
  Eroare   : 4.496718e-05
----------------------------------------
Test 3: cadranul 3 (-135 grade)
  X=-10000 Y=-10000
  CORDIC Rad: -2.356150e+00
  REAL   Rad: -2.356194e+00
  Eroare   : 4.496718e-05
----------------------------------------
```

## 5.1 Comparatie cu IP-CORE Xilinix




## IP-CORE – Xilinix

| | **Word Serial** (Mic/Lent) | **Parallel** (Mare/Rapid) |
|---|---|---|
| Slice LUTs | **~350 - 500** | ~2.500 - 3.200 |
| Slice Registers | ~350 - 450 | ~3.500 - 4.500 |
| Bonded IOB | ~96 - 105 | ~96 - 105 |
| BUFGCTRL | **1** | 1 |
| Latență | 32 - 35 cicli | |
| | | |

## 6.Concluzii.

Rezultatele prezentate în tabelul de mai sus sunt doar aproximative, preluate de pe internet. Totuși pe baza lor putem să prezentăm câteva concluzii.

Metodele de calcul prezentate în lucrarea de față sunt metode simple care nu au o precizie foarte mare. Se poate observa faptul că în rezultatele modulului cordic prezentat, precizia este de 3-4 zecimale.

Același precizie este și în cazul modulului implementat pe bază de Look-Up-Table. În aplicațiile moderne, această precizie este foarte slabă. Ip-core Xilinix oferă o precizie de mai mult de 10 zecimale. Această diferență de precizie se va reflecta în alegerile de proiectare pe care le va face inginerul. Din datele mai sus prezentate observăm că modulele prezentate în această lucrare folosesc mai puține elemente FPGA decât Ip-Core Xilinix. Dar de asemenea viteza este mai scăzută, și preciza este mult sub modulele Xilinix. Astfel proiectantul va tinde mereu să facă compromisuri de proiectare în funcție de necesitățile proprii. Un compromis fundamental este între performanță și resurse. Dacă va crește performanța și precizia de calcul, de cele mai multe ori vor fi necesare mai multe resurse și viceversa.

Astfel activitatea de proiectare a sistemelor de calcul de mare precizie vine cu multe probleme dificile, multe compromisuri între resurse și performanță, vine cu noi idei și metode ingenioase de a calcula diferite funcții care pot să revoluționeze știința calculatoarelor. Procesul este unul fascinant pentru aceia care se încumetă la noi provocări.